Results in Control and Optimization 4 (2021) 100030

Contents lists available at ScienceDirect

Results in Control and Optimization

journal homepage: www.elsevier.com/locate/rico

# Reinforcement learning for fluctuation reduction of wind power with energy storage

Zhen Yang [a], Xiaoteng Ma [a], Li Xia [b,*], Qianchuan Zhao [a], Xiaohong Guan [a,c]

[a] Center for Intelligent and Networked Systems (CFINS), Department of Automation, Tsinghua University, Beijing 100084, China
[b] Business School and the Guangdong Province Key Laboratory of Computational Science, Sun Yat-Sen University, Guangzhou 510275, China
[c] MOE KLINNS Lab, Xi'an Jiaotong University, Xi'an 710049, China

ARTICLE INFO

*Keywords:*
Wind power
Battery energy storage
Fluctuation reduction
Sensitive-based optimization
Reinforcement learning

ABSTRACT

The intrinsic randomness of renewable energy has a negative impact on the safety of power grid. In this paper, we aim at decreasing large fluctuations of the power output from a wind farm integrated with a battery energy storage system (BESS), so as to improve the stability and quality of the power system. The control method is to dynamically charge or discharge the BESS, coordinated with limited wind curtailment. The fluctuation of total power output is measured by variance, which reflects the risk to the safety of grid. The difficulty is that this dynamic optimization problem does not meet the requirement of a standard Markov decision process (MDP) model, since the variance metric is not additive. To solve this problem, we first propose the sensitivity-based optimization method and derive a difference formula to quantify the variance metrics of the system. Then we implement the optimization approach as reinforcement learning algorithms, in a mode of data-driven. We develop the Q-learning algorithm so that it can be executed online and generate improved policies repeatedly with observed data. Furthermore, we implement Deep Q-Networks (DQN) to handle the difficulty of continuous states. The performance of the proposed algorithms is verified with real data, which demonstrates the effectiveness of our algorithms.

## 1. Introduction

Renewable energy technologies are developing very fast in recent years, which provide a solution to a global sustainable energy future. It is reported that China plans to make renewable energy account for 35 percent of the total electricity consumption by 2030 [1]. Making good use of the renewable energy is an increasingly important issue in power systems. Wind power is one of the most popular renewable energy sources. Nonetheless, power fluctuations caused by the intermittency and uncertainty of the wind power deteriorate the grid, thus the need for reducing drastic wind power fluctuations has been heightened. See [2] for the harm caused by short-term fluctuations and [3–5] for that caused by long-term fluctuations. In the long-run view, stable renewable sources could reduce the energy reservation of traditional generators, and help to maintain economy schedules by the main grids [6].

To smooth short-term fluctuations of the wind power, attempts have been made in the literature by enhancing the function of wind turbines and utilizing the inertia energy [7]. Also, there has been an increasing interest in the utilization of energy storage systems (ESSs) [8], and a large amount of algorithms have been developed to schedule the ESS. One of the most commonly utilized algorithms are filtering-based algorithms [9,10]. Other popular algorithms include two-stage programming [11] and model predictive control (MPC) [3], etc.

* Corresponding author.
*E-mail address:* xiali5@sysu.edu.cn (L. Xia).

https://doi.org/10.1016/j.rico.2021.100030
Received 24 March 2021; Received in revised form 14 May 2021; Accepted 19 June 2021
Available online 25 June 2021
2666-7207/© 2021 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

**Nomenclature**

**Indices and Sets**

| | |
|---|---|
| $t$ | Index of time epochs. $t = 1, 2, \ldots, \infty$ |
| $k$ | Index of hour in one day. $k = 1, 2, \ldots, N_T$ |
| $\boldsymbol{S}$ | Markov process. $\boldsymbol{S} = \{S_t, t = 1, 2, \cdots, \infty\}$ |
| $\mathcal{S}$ | State space of the Markov process. |
| $\mathcal{A}$ | Action space of the Markov process |
| $\mathcal{U}$ | Policy space of the Markov process |

**Parameters**

| | |
|---|---|
| $N_W$ | Number of wind power states |
| $N_B$ | Number of battery level states |
| $N_T$ | Number of hours in one day |
| $N$ | Number of system states |
| $W_{cap}$ | Wind power capacity |
| $W_{cur}$ | Maximal curtailment power |
| $W_{std}$ | Standard power output |
| $V_{cutin}$ | Wind turbine cut-in speed |
| $V_{cutout}$ | Wind turbine cutout speed |
| $V_{rated}$ | Wind turbine rated speed |
| $B_{min}$ | Minimal battery level |
| $B_{max}$ | Maximal battery level |
| $B_{cap}$ | Battery capacity |
| $C_{max}$ | Maximal charging power of battery |
| $D_{max}$ | Maximal discharging power of battery |
| $\alpha$ | Learning rate in Q-learning |
| $\beta$ | Smooth factor in Q-learning |
| $\gamma$ | Discount factor in Q-learning |
| $\epsilon$ | Exploration factor in Q-learning |
| $n$ | Batch size in Q-learning |
| $M_{eval}$ | Episodes for evaluation in Q-learning |
| $T_{update}$ | Updating steps in Q-learning |
| $T_{target}$ | Target replacing steps in Q-learning |

**Variables**

| | |
|---|---|
| $S$ | System state |
| $a^{[1]}$ | Action of battery operation |
| $a^{[2]}$ | Action of wind curtailment operation |
| $A$ | Action. $A_t := (a_t^{[1]}, a_t^{[2]})$, and $A_t \in \mathcal{A}$. |
| $b$ | Battery level |
| $w$ | Wind power output |
| $v$ | Wind speed |
| $p$ | Probability of transition between states |
| $\boldsymbol{P}_w$ | Wind power transition matrix |
| $\boldsymbol{P}$ | State transition matrix |
| $\mu$ | Scheduling policy |

Mostly the scheduling of the wind power is integrated with the unit commitment (UC) and economic dispatch (ED) of traditional generators, in which the wind power can be formulated by statistical models [12–15] or be predicted [16,17]. However, dynamic processes of wind power and the related optimization issues, such as long-term fluctuation reduction, have seldom been studied. The main difficulty lies on the challenge of long-term forecasting, which may be addressed by rule-based methods to some extent. In [18] and [19], the long-term scheduling is achieved by rule-based methods which provide battery storage scheduling commands with the support of expertise, avoiding the inaccuracy of long-term forecasting. In [20], the fuzzy logic control (FLC) is utilized for

| | |
|---|---|
| $r$ | Reward in MDPs |
| $\boldsymbol{\pi}$ | Steady-state probability vector |
| $\eta$ | Long-term average power output |
| $\eta_\sigma$ | Long-term variance of power output |
| $V$ | Fluctuation potential |
| $Q$ | Action-value function of long-term variance |
| $\mathcal{B}$ | Replay buffer |
| $\theta$ | Coefficients of neural network in deep Q-learning |
| **Function** | |
| $W(\cdot)$ | Wind turbine output function |

minimizing the tie-line power fluctuations caused by the wind power, in which the scheduling orders of the battery storage depend on several predetermined rules. Though the rule-based policies work in practice, they barely exploit the stochastic characteristics of the system with new data.

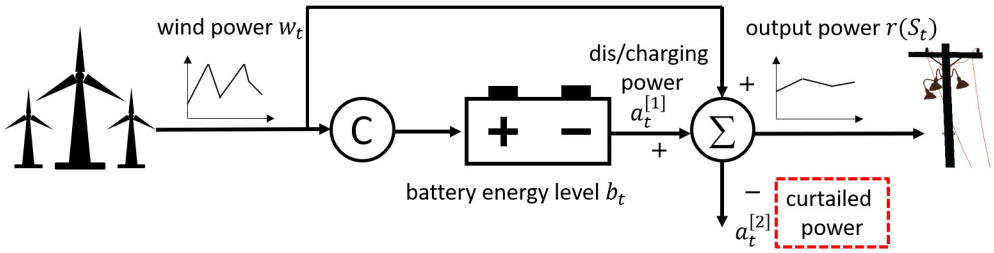
One of the motivations of our study is developing dynamic optimization algorithms considering the stochastic characteristics of the wind power, to reduce the long-term fluctuation of the wind power output by scheduling the ESS. Dynamic programming (DP) is a natural candidate to achieve long-term scheduling by exploiting the stochastic model of the wind power, and variance as an indicator of risk is suitable for depicting the power fluctuation in DP. Variance can be combined with mean to seek a balance between economy and safety, as applied in literature [21–23]. These studies mainly focus on the variance of cumulative profit of wind power generation, rather than the long-term variance of wind power during a dynamic process.

A nonnegligible difficulty in the application of DP where variance is used as the metric is that the optimization criterion is not Markovian, due to the fact that variance or other higher moment of random variables are non-linear and non-additive, resulting in a violation of the principle of time consistence in DP [24,25]. The sensitivity-based optimization can be employed as a means of addressing this difficulty by changing the variance minimization problem into a standard Markov decision process (MDP), enabling the application of DP and policy iteration [24–26]. A sensitivity-based policy iteration algorithm has been studied in [6], which provides the optimal solution of fluctuation reduction in a wind farm with BESS. But this study has not dealt with the scenario in which wind power curtailment is included. Moreover, policy iteration algorithms are implemented offline and they rely on the knowledge of system models. The effectiveness of the policy iteration will be reduced in the absence of the characteristics of system models.

The other important motivation of our study is to relieve the ineffectiveness caused by the policy iteration with model-free algorithms. Data-driven online learning algorithms are promising approaches of determining the optimal policy with real data. In this research area, reinforcement learning (RL) is one of the most prevalent algorithms [27], thus is specifically suitable for long-term wind power fluctuation reduction. RL is able to improve the performance of conventional controllers such as PID [28] or provide innovative economic dispatch schemes in power systems [29], as it can address the randomness caused by renewable energy or loads and provide model-free solutions without the construction of stochastic models in advance. RL has been commonly utilized in power systems, such as electric vehicles scheduling [30], demand response [31], building control [32], energy storage control [33] and uncertainty models [34,35]. Q-learning is a popular RL algorithm, but it suffers from the 'curse of dimensionality' when the state space and action space are large or continuous. To handle the 'curse of dimensionality', deep Q-networks (DQN) can be leveraged to address the continuous state-space problems. DQN is a combination of RL and deep neural networks, which provides flexible Q-function approximation [36]. Although there are papers focusing on tackling the uncertainties of renewable energy with ESSs using RL, see [37,38], they are usually oriented by economic indicators, which are first-order metrics. Economic indicators are important and common in power systems, but in this paper we study the scheduling problem from the perspective of the variance, which is seldom studied but potentially effective for both the safety and economy of the grid as we discussed above. Hence, we study Q-learning algorithms which are promising for solving the problem of long-term wind power fluctuation reduction, as they require no prior knowledge of the system model and can be implemented both offline and online.

The major content of this paper is to study the long-term wind power fluctuation reduction problem in a wind farm with BESS. Specifically, we choose the BESS due to the rapid response and high power density of the battery storage. Different from classical mean-based systems, we instead take account of higher order moments of random variables. We regard the variance of the total power output as the metric of power fluctuation to establish a series of approaches. The objective is to obtain the optimal scheduling policy of charging or discharging the BESS, cooperated with limited wind curtailment operation. In order to tackle the problem that the variance as the cost function in Markov systems is not additive, we develop an iterative optimization algorithm by deriving a variance difference formula based on the sensitivity-based optimization theory. Moreover, we develop a Q-learning method by applying the sensitivity-based optimization theory as well, so that we can implement this algorithm in a data-driven mode. Furthermore, we develop a DQN style algorithm to handle the difficulty of continuous states, which overcomes the 'curse of dimensionality' and is able to implement in practice.

The main contribution of this paper is the effective approaches for reducing the long-term fluctuation (measured by variance) of wind power output with the BESS while considering wind curtailment. Variance-related criterion is difficult to be handled by




$$\min\{\sigma^2[r(S_t)]\}\ s.t.\ r(S_t) = w_t - \left(a_t^{[1]} + a_t^{[2]}\right); b_{t+1} = b_t + a_t^{[1]}$$

**Fig. 1.** System structure of the wind farm integrated with BESS.

traditional dynamic programming methods. Our approach with sensitivity-based optimization provides a new perspective to handle the long-run variance metrics. The reinforcement learning algorithms developed in this paper are not applied directly from existing literature, but based on the difference formula in sensitivity-based optimization theory which fits the very dynamic optimization problem. To the best of our knowledge, our work is the first one to develop reinforcement learning algorithms to optimize the variance of renewable power output with BESS in dynamic processes.

The remainder of this paper is organized as follows. We describe the system model and the problem formulation of long-term fluctuation reduction in Section 2. The main results and the optimization algorithms are shown in Section 3. Numerical experiments based on real data are presented in Section 4 to demonstrate our results, followed by the conclusions in Section 5.

## 2. Problem formulation

### 2.1. System structure

In this paper, we consider a wind farm with a BESS, out of which the wind power generated is sent to the main grid. Fig. 1 demonstrates the system structure. The wind power output fluctuation is reduced by implementing battery charging or discharging operation and appropriate wind curtailment, while setting an upper limit on the amount of wind curtailment.

### 2.2. System constraints

The system state includes the wind power, the battery level and the hour index in one day. In a discrete event dynamic system, each state has a corresponding time, indicating the time epoch in which the variable is. The wind power from the wind farm at time epoch $t$ and hour $k$ is a random variable, noted as $w_t(k) \subseteq \{1, 2, 3, \dots, N_W\}$. The battery level at stage $t$ and hour $k$ is $b_t(k) \subseteq \{1, 2, 3, \dots, N_B\}$. Besides, one day is divided into 24 stages by hour denoted with $k = 1, 2, \dots, N_T$, and naturally $N_T = 24$. For the sake of simplicity, we write $w_t(k)$ and $b_t(k)$ as $w_t$ and $b_t$, respectively. The relationship between the wind power and wind speed [39] is shown as:

$$W(v_t) = \begin{cases} W_{cap}, & V_{rated} \le v_t < V_{cutout} \\ W_{cap}(\frac{v_t}{V_{rated}})^3, & V_{cutin} \le v_t < V_{rated} \\ 0, & \text{others.} \end{cases} \tag{1}$$

The scheduling action includes the charging or discharging of the battery storage and wind curtailment operation. We set $a^{[1]}$ as the battery operation and $a^{[2]}$ as the wind curtailment operation. We define $a_t^{[1]} := b_{t+1} - b_t$, which can be either positive or negative, representing the direction of power flow with respect to our charging or discharging operation. Action $a^{[2]}$ is either larger than or equal to zero, which means the amount of wind curtailment.

In the process of system state transitions, the dynamics and constraints can be concluded below:

$$0 \le w_t \le W_{cap}, \tag{2}$$

$$B_{min} \le b_t \le B_{max}, \tag{3}$$

$$b_{t+1} = b_t + a_t^{[1]}, \tag{4}$$

$$-D_{max} \le a_t^{[1]} \le \min(C_{max}, w_t), \tag{5}$$

$$0 \le a_t^{[2]} \le \min(W_{cur}, w_t), \tag{6}$$

$$k_{t+1} = k_t \% N_T + 1, \tag{7}$$

where $t = 1, 2, \dots, \infty$, $k = 1, 2, \dots, N_T$.

Constraint (2) shows the range of the wind power generated. (3) and (4) are constraints of battery level and its change, respectively. (5) means that the charging and discharging rate should be confined by the maximal ability and the wind power generated. (6) ensures that the curtailed amount of wind power does not overflow. (7) shows the circulation of hour $k_t$ in one day at time stage $t$, where $'\%'$ is modulo operator.

We assume that the battery energy level varies linearly and there is no self-discharging. These assumptions have been made in work about BESS [40], which are for the benefit of building simulation models. The maximum depth of discharge (DOD) should be considered in practice to prolong the life of batteries, and our experiments allow for this factor by assuming the battery capacity as the available capacity after considering the DOD in simulations.

## 3. Solution methodology

### 3.1. Policy iteration

We define the system state as $S_t := (w_t, b_t, k_t)$, where $t = 1, 2, \dots, \infty$ is the time epoch index. We define a Markov process $\boldsymbol{S} = \{S_t, t = 1, 2, \dots, \infty\}$. The state space is $\mathcal{S} = \{1, 2, \dots, N\}$, where $N$ is the product of $N_W$, $N_B$ and $N_T$. The action is defined as $A_t := (a_t^{[1]}, a_t^{[2]})$, and $A_t \in \mathcal{A}$, where $\mathcal{A}$ is the action space. The state space and the action space are demonstrated in detail in the Appendix for simulation experiments. Tables A.1 to A.4 present the discrete cases, and Tables A.5 and A.6 present the continuous cases.

By collecting real wind speed data, we can build the probability transition matrices of the wind power, then the system. We use the data sets from the Measurement and Instrumentation Data Center (MIDC) in the National Renewable Energy Laboratory (NREL) [41]. The data of wind speed is in hourly basis from 0:00 September 1, 2005 to 23:59 September 1, 2020. The wind speed data is from instruments on meteorological tower located at $39^{\circ}54'38.34''$N, $105^{\circ}14'5.28''$W, and about 1855 meters above mean sea level. The probability of transition from state $m$ to state $n$ at hour $k$ can be calculated as below.

$$p_{mn}^{(k)} := \frac{N_{mn}^{(k)}}{N_m^{(k)}}, \tag{8}$$

where $N_{mn}^{(k)}$ is the number of transitions from state $m$ to state $n$ at hour $k$, and $N_m^{(k)}$ is the number of occurrences of state $m$ at hour $k$. Then the transition matrix from hour $k$ to hour $k+1$ can be calculated as

$$\boldsymbol{P}_w^{(k)} = \begin{pmatrix} p_{11}^{(k)} & \cdots & p_{1n}^{(k)} & \cdots & p_{1N_W}^{(k)} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ p_{m1}^{(k)} & \cdots & p_{mn}^{(k)} & \cdots & p_{mN_W}^{(k)} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ p_{N_W 1}^{(k)} & \cdots & p_{N_W n}^{(k)} & \cdots & p_{N_W N_W}^{(k)} \end{pmatrix}, \tag{9}$$

where $k = 1, 2, \dots, N_T$ and $m, n = 1, 2, \dots, N_W$. The wind power probability transition matrix $\boldsymbol{P}_w$ and the system probability transition matrix $\boldsymbol{P}$ can then be established by calculating the historical transitions, as well as considering the dynamics and constraints of the system, in the form of

$$\boldsymbol{P}_w = \begin{pmatrix} \boldsymbol{0} & \boldsymbol{P}_w^{(1)} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{P}_w^{(2)} & \cdots & \boldsymbol{0} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \boldsymbol{0} & \boldsymbol{0} & \dots & \boldsymbol{0} & \boldsymbol{P}_w^{(N_T-1)} \\ \boldsymbol{P}_w^{(N_T)} & \boldsymbol{0} & \boldsymbol{0} & \dots & \boldsymbol{0} \end{pmatrix}, \tag{10}$$

and

$$\boldsymbol{P} = \begin{pmatrix} \boldsymbol{0} & \boldsymbol{P}^{(1)} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{P}^{(2)} & \cdots & \boldsymbol{0} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \boldsymbol{0} & \boldsymbol{0} & \dots & \boldsymbol{0} & \boldsymbol{P}^{(N_T-1)} \\ \boldsymbol{P}^{(N_T)} & \boldsymbol{0} & \boldsymbol{0} & \dots & \boldsymbol{0} \end{pmatrix}, \tag{11}$$

respectively. Similar work has been done in [6].

The sensitivity-based optimization theory builds a framework where the performance of different policies can be compared by the difference formula, and then iterative optimization algorithms can be developed with the difference formula [26]. This characteristic helps to tackle nonlinear metrics when the Bellman equation does not hold. To derive the difference formula, we start with some definitions. We define the reward function as the total power output $r(S_t) = w_t - (a_t^{[1]} + a_t^{[2]})$. The long-term average power output is defined as

$$\eta = \lim_{T\to\infty} \frac{1}{T} \mathbb{E}\left\{ \sum_{t=0}^{T-1} r(S_t) \right\} = \boldsymbol{\pi r}, \tag{12}$$

where $\boldsymbol{\pi}$ represents the steady-state probability as an $N$-dimensional column vector $\boldsymbol{\pi} := (\pi(1), \pi(2), \ldots, \pi(N))$. The probability of the Markov system being at state $s$ is denoted by $\pi(s)$, where $s \in \mathcal{S}$. Also, $\eta$ is independent of the initial state $S_0$, as the system is assumed ergodic. The steady-state probability $\boldsymbol{\pi}$ can be calculated by using the Cesaro limit [26] as follows :

$$\boldsymbol{P}^* = \lim_{N\to\infty} \frac{1}{N} \Sigma_{n=0}^{N-1} \boldsymbol{P}^n, \tag{13}$$

where any row of $\boldsymbol{P}^*$ represents $\boldsymbol{\pi}$.

On the basis of the definition of $\eta$, we define the long-term variance of power output as

$$\eta_\sigma = \lim_{T\to\infty} \frac{1}{T} \mathbb{E}\left\{ \sum_{t=0}^{T-1} (r(S_t) - \eta)^2 \right\}, \tag{14}$$

which shows the severity of the power output fluctuations. For a stationary and deterministic policy $\mu$ within the policy space $\mathcal{U}$, it is able to provide certain action orders in different states, denoted by $\mu(s)$. We aim at finding an optimal policy under which the long-term variance $\eta_\sigma$ can be minimized.

Referring to the sensitivity-based optimization theory, we define the *power output fluctuation potential* as below.

$$V(s) := \lim_{T\to\infty} \mathbb{E}\left\{ \sum_{t=0}^{T-1} \left[(r(S_t) - \eta)^2 - \eta_\sigma\right] \mid S_0 = s \right\}. \tag{15}$$

Also, we define an $N$-dimensional column vector $\boldsymbol{V}$ with the element being $V(s)$, $s \in \mathcal{S}$. The fluctuation potential is a quantity measuring the long-term cumulative deviation of $(r(S_t) - \eta)^2$ away from $\eta_\sigma$. If $V(s)$ is positive and large, it means that the initial state $s$ is at a position with high fluctuation potentials compared to the long-term variance of power output $\eta_\sigma$. In classical MDP theory [42], the concept of $\boldsymbol{V}$ has a similar meaning of *relative value function*. Therefore, the *Poisson equation* can be derived as

$$\boldsymbol{V} = (\boldsymbol{r} - \eta\boldsymbol{e})_\odot^2 - \eta_\sigma \boldsymbol{e} + \boldsymbol{P}\boldsymbol{V}, \tag{16}$$

where $(\boldsymbol{r} - \eta\boldsymbol{e})_\odot^2 := (((r(1) - \eta)^2), ((r(2) - \eta)^2), \ldots, ((r(N) - \eta)^2))^T$, and $\boldsymbol{e}$ is an $N$-dimensional column vector with each element being 1. We can either calculate $\boldsymbol{V}$ by referring to the method in [43] or estimate it according to its definition in Eq. (15) using sample paths $\boldsymbol{S}$. For more details on online estimation and learning, we can refer to the book [26]. In this paper, we can obtain $\boldsymbol{V}$ as

$$\boldsymbol{V} = (\boldsymbol{I} - \boldsymbol{P} + \boldsymbol{e}\boldsymbol{\pi})^{-1} (\boldsymbol{r} - \eta\boldsymbol{e})_\odot^2, \tag{17}$$

which has been illustrated in [43].

Furthermore, we study the difference of the long-term variance when the policy varies from $\mu$ to $\mu'$, where $\mu, \mu' \in \mathcal{U}$. The variance of total power output under policy $\mu$ and $\mu'$ can be derived respectively as

$$\eta_\sigma = \boldsymbol{\pi}(\boldsymbol{r} - \eta\boldsymbol{e})_\odot^2, \tag{18}$$

$$\eta'_\sigma = \boldsymbol{\pi}'(\boldsymbol{r}' - \eta'\boldsymbol{e})_\odot^2. \tag{19}$$

Then the difference between equation (18) and (19) can be derived as [24]. We have

$$\eta'_\sigma - \eta_\sigma = \boldsymbol{\pi}'[(\boldsymbol{P}' - \boldsymbol{P})\boldsymbol{V} + (\boldsymbol{r}' - \eta\boldsymbol{e})_\odot^2 - (\boldsymbol{r} - \eta\boldsymbol{e})_\odot^2] - (\eta' - \eta)^2. \tag{20}$$

The difference formula (20) quantifies the fluctuation difference between any two policies in the Markov system. It shows the relation between the fluctuation degree and the corresponding policy, where different policy has different $\boldsymbol{P}$ and $\boldsymbol{r}$. A new policy with $\boldsymbol{P}'$ and $\boldsymbol{r}'$ which ensures that $\boldsymbol{\pi}'[(\boldsymbol{P}' - \boldsymbol{P})\boldsymbol{V} + (\boldsymbol{r}' - \eta\boldsymbol{e})_\odot^2 - (\boldsymbol{r} - \eta\boldsymbol{e})_\odot^2] < 0$ can guarantee that $\eta'_\sigma - \eta_\sigma < 0$, as $-(\eta' - \eta)^2$ is always non-positive. With the fact that the long-term variance of total power output under policy $\mu'$ is less than that under policy $\mu$, the performance of fluctuation reduction will be improved under the new policy $\mu'$. We can continually derive better policies which make the fluctuation smaller by policy iteration.

Given a finite policy set $\mathcal{U}$, a policy iteration algorithm which minimizes the fluctuation of total power output can be derived based on the difference formula. We derive the sensitivity-based policy iteration algorithm for fluctuation reduction shown as Algorithm 1. Eq. (21) is the key step in Algorithm 1, which means that in each iteration we select the action that would minimize the sum of instant 'variance', denoted by $[r(s,a) - \eta^{(l)}]^2$, and long-term variance potential of the next state, denoted by $[\sum_{s'\in\mathcal{S}} p^a(s,s')V^{(l)}(s')]$. The use of Eq. (21) is based on the difference formula (20) which illustrates the principle of deriving better policies.

### 3.2. Tabular Q-learning

In this subsection, we extend the idea of policy iteration to Q-learning, which can be conducted online. For all the efficiency of policy iteration in theory, the implementation of policy iteration needs the transition probability matrix $\boldsymbol{P}$, which is hard to obtain in most real cases. Besides, once $\boldsymbol{P}$ is constructed, extra data cannot be used directly for better policies. Hence, we develop an online and model-free algorithm in which the policy is updated with new data from an interactive environment. We discuss the tabular cases in this part, and move on to the continuous state space in the next subsection.

**Algorithm 1** Sensitivity-based Policy Iteration for Fluctuation Reduction

1: Initialize the initial scheduling policy $\mu^{(0)} \in \mathcal{U}$, and set iteration number $l = 0$.
2: For policy $\mu^{(l)}$, calculate the steady state probability distribution $\boldsymbol{\pi}^{(l)}$, the long-term average power output $\eta^{(l)}$, and the fluctuation potential $\boldsymbol{V}^{(l)}$.
3: Check the feasible action set for each state. Update the new policy as

$$\mu^{(l+1)}(s) := \text{argmin}_{a\in\mathcal{A}}\left\{\sum_{s'\in\mathcal{S}} p^a(s,s')V^{(l)}(s') + [r(s,a)-\eta^{(l)}]^2\right\}, \forall s \in \mathcal{S}, \tag{21}$$

where $s'$ is the state of the next time epoch. In this step, keep $\mu^{(l+1)}(i) = \mu^{(l)}(i)$ if possible to avoid policy oscillation phenomenon.

4: **if** $\mu^{(l+1)} = \mu^{(l)}$ **then**
5: Set the optimal policy $\mu^* = \mu^{(l)}$, and stop.
6: **end if**
7: Set $l := l + 1$ and go back to step 2.

The definition of the long-term average performance and the variance of the Markov chain are the same as that in policy iteration, shown in Eqs. (12) and (14) respectively. As discussed in the last subsection, if we set the estimated performance as a constant, it will turn out to be a standard MDP to optimize the pseudo variance [24,25]. Methods for solving the standard MDP can be implemented to this problem directly.

The action-value function of long-term variance of the total power output for any policy $\mu$ is defined as

$$Q_\sigma^\mu(s,a) = \mathbb{E}_\mu\left\{\sum_{t=0}^{\infty}\left[(r(S_t,A_t)-\eta)^2 - \eta_\sigma\right] \mid S_0 = s, A_0 = a\right\}, \tag{22}$$

which means the potential of power output fluctuation under current policy $\mu$. We have $\eta$ and $\eta_\sigma$ fixed for a predetermined period of time in the Q-learning algorithm, then update them with new data. When $\eta$ and $\eta_\sigma$ remain unchanged, the *Bellman equation* can be derived directly as

$$Q_\sigma^\mu(s,a) = \mathbb{E}_\mu\left\{(r(s,a)-\eta)^2 - \eta_\sigma + Q_\sigma^\mu(s',a')\right\}, \tag{23}$$

where $s', a'$ are the state and action of the next time epoch, sampled by policy $\mu$. Then we can derive a so-called *Bellman local optimality equation* as

$$Q_\sigma^*(s,a) = \mathbb{E}\left\{(r(s,a)-\eta^*)^2 - \eta_\sigma^* + \min_{a'} Q_\sigma^*(s',a')\right\}, \tag{24}$$

which is the basis of our Q-learning algorithm.

The derivation of the *Bellman local optimality equation* is similar to that in literature [24] and [25]. We derive the Q-learning algorithm under average criterion as shown in Algorithm 2. See [44] for more rigorous convergence analysis of Q-learning for average cost control. The whole sample path for training is generated by a random initial state and the transition probability matrix $\boldsymbol{P}$. To increase the diversity of the simulation paths, we break the whole path into $M$ pseudo episodes, and reset the system state every $T$ steps. We evaluate the performance every $M_{eval}$ episodes, and smooth the estimation of $\eta, \eta_\sigma$ with moving average method. Provided the transition tuple $(S_t, A_t, r(S_t), S_{t+1})$ collected by $\epsilon$-greedy strategy, we update the table $Q_\sigma(s,a)$ according to the Bellman local optimality equation.

### 3.3. Deep Q-learning

Besides the Q-learning discussed in last subsection, we further study the variance reduction problem with a continuous state space. The states of both wind power and the battery level are continuous numbers in practice, so it is necessary to extend Q-learning to deep Q-learning which is capable of handling continuous-state problems.

Different from the policy iteration in tabular form and Q-learning, we define the continuous system state $S_t := (w_t, b_t, k_t)$, where $w_t \in [0, W_{cap}]$ and $b_t \in [B_{min}, B_{max}]$ are the wind power output and battery level at time stage $t$, respectively. $k_t$ is the hour index of a day. The action is also defined as $A_t := (a_t^{[1]}, a_t^{[2]})$, $A_t \in \mathcal{A}$, where $\mathcal{A}$ as the state space is the same with that in tabular cases. Rather than directly enter the system state into the neural network, we adjust the input to $S_t := \left(w_t, b_t, cos(k_t/24), sin(k_t/24)\right)$ for capturing the periodic feature of the hour index $k$.

The deep Q-learning style algorithm can be derived with the help of deep neural network. Instead of using a big table for storage, we use a neural network to approximate the Q-function defined in Eq. (22). There have been a number of studies on improving the performance of DQN, and we adopt some in our method. We use the target network to keep the target value fixed in a training period and use an experience replay buffer to break the timing relationship between training samples [45]. To handle the problem of overestimation caused by the maximization operation (minimization in the paper), we apply the double Q-learning to differ the networks for choosing target action and getting target value [46]. We also use dueling network [47] to get better performance with a large number of actions.

**Algorithm 2** Tabular Q-learning for Fluctuation Reduction

Initialize the table $Q_\sigma(s,a)$ randomly
**for** $m = 1 : M$ **do**
  **if** $m$ mod $M_{eval}$ **then**
    Reset the system and receive initial state $S_1$
    **for** $t = 1 : T_{eval}$ **do**
      Select $A_t = \arg\min_a Q_\sigma(S_t, a)$
      Execute action $A_t$, observe $r(S_t), S_{t+1}$
    **end for**
    Set $\hat{\eta} = \frac{1}{T_{eval}} \sum_{t=1}^{T_{eval}} R_t$
    Set $\hat{\eta_\sigma} = \frac{1}{T_{eval}} \sum_{t=1}^{T_{eval}} (R_t - \hat{\eta})^2$
    $\eta \leftarrow \beta\hat{\eta} + (1-\beta)\eta$, $\eta_\sigma \leftarrow \beta\hat{\eta_\sigma} + (1-\beta)\eta_\sigma$
  **end if**
  Reset the system and receive initial state $S_1$
  **for** $t = 1 : T$ **do**
    With probability $\epsilon$ select a random action $A_t$
    otherwise select $A_t = \arg\min_a Q_\sigma(S_t, a)$
    Execute action $A_t$, observe $r(S_t), S_{t+1}$
    Set $y_t = (r(S_t) - \eta)^2 - \eta_\sigma + \min_{a'} Q'_\sigma(S'_t, a')$
    Update $Q_\sigma(S_t, A_t) \leftarrow (1-\alpha)Q_\sigma(S_t, A_t) + \alpha y_t$
  **end for**
**end for**

With the ability to handle continuous-state problems, we directly use the real data to train the deep Q-learning, meaning that we use the real wind power profile as the input of the simulation. To increase the data diversity and make full use of the data in hand, we break the whole data set into pieces for training. We randomly choose a start point to reset the system state and break the timing relationship when the episode terminates. Similar to the tabular Q-learning, we use $\epsilon$-greedy policy to select actions so that all possible actions can be chosen randomly, and we estimate current $\eta, \eta_\sigma$ every $M_{eval}$ episodes with greedy policy to help evaluate and update the Q networks. The transition tuple $(S_t, A_t, r(S_t), S_{t+1})$ collected from the simulation environment is pushed into a replay buffer $\mathcal{B}$ and sampled periodically to update the network parameters with back-propagation. The main network is updated every $T_{update}$ steps and the target network is replaced by the main network every $T_{target}$ steps. The full work-flow is shown in Algorithm 3.

### 3.4. Comparison of algorithms

The comparison of the three proposed algorithms is shown in Fig. 2. In the sensitivity-based policy iteration algorithm, the transition probability matrix is derived by historical data and system constraints, and the fluctuation potential $V(s)$ can be directly calculated by the Poisson equation in (16). The fluctuation potential is updated after each iteration until the algorithm converges and we get the optimal policy. As an offline algorithm, its advantage is that when the system transition probability matrix is available, the optimal policy can be calculated quickly. Its disadvantage is that the new data cannot be used immediately thus the policy cannot be updated online.

However, when historical data is insufficient or incomplete, the transition probability matrix cannot be derived. For example, a newly built wind farm or a wind farm without historical observations will not be a proper case for applying the policy iteration algorithm. In this circumstance, Q-learning is applicable. The Q-learning demonstrates how the algorithm works when all variables are discrete. The values of Q-function for every state–action pair $(s,a)$, which cannot be calculated with the system structure, are stored in a table and constantly evaluated and updated with the continuous interaction between the agent and environment until the algorithm converge. As an online algorithm, its performance by the sample path gradually improves towards the optimal value when new data is available.

Furthermore, the deep Q-learning offers a practical way of dealing with large state spaces, even continuous spaces with an infinite number of states. Instead of storing the values of Q-function for every state–action pair $(s,a)$, we use parametric neural networks to estimate the values of Q-function, and update the parameters of the neural network, denoted as $\theta$. Although it is assumed that the training data are independently and uniformly distributed during the process of training, there is always a correlation between the data collected through Q learning, which is easy to cause the instability of the neural network. To address the correlation of data, the replay buffer can be applied so that each training sample will come from multiple interaction sequences and the fluctuation of a single sequence can be alleviated, which makes the training effect more stable.

Some issues about the reliability of implementing RL algorithms have been raised. One concern, e.g., is that during the training of the first few weeks, the operation of exploration may harm the system. We deal with this problem by setting feasible initial policies and implementing simulation experiments with historical data. To test the performance of these three algorithms and make comparison, we conduct numerical experiments with real historical data of wind power. The results and discussions are shown below in the next section.

**Algorithm 3** Deep Q-learning for Fluctuation Reduction

Initialize the network $Q_\sigma(s, a; \theta)$ with weights $\theta$ randomly
Initialize the target network $Q'_\sigma(s, a; \theta')$ with weights $\theta' \leftarrow \theta$
Initialize replay buffer $\mathcal{B}$ using random policy
**for** $m = 1 : M$ **do**
  **if** $m$ mod $M_{eval}$ **then**
    Reset the system and receive initial state $S_1$
    **for** $t = 1 : T_{eval}$ **do**
      Select $A_t = \arg\min_a Q_\sigma(S_t, a; \theta)$
      Execute action $A_t$, observe $r(S_t), S_{t+1}$
    **end for**
    Set $\hat{\eta} = \frac{1}{T_{eval}} \sum_{t=1}^{T_{eval}} R_t$
    Set $\hat{\eta_\sigma} = \frac{1}{T_{eval}} \sum_{t=1}^{T_{eval}} (R_t - \hat{\eta})^2$
    $\eta \leftarrow \beta\hat{\eta} + (1-\beta)\eta$, $\eta_\sigma \leftarrow \beta\hat{\eta_\sigma} + (1-\beta)\eta_\sigma$
  **end if**
  Reset the system and receive initial state $S_1$
  **for** $t = 1 : T$ **do**
    With probability $\varepsilon$ select a random action $A_t$
    otherwise select $A_t = \arg\min_a Q_\sigma(S_t, a; \theta)$
    Execute action $A_t$, observe $r(S_t), S_{t+1}$
    Store transition $(S_t, A_t, r(S_t), S_{t+1})$ in $\mathcal{B}$
    **if** $t$ mod $T_{update}$ **then**
      Sample a random mini batch of $n$ transitions
      $(s_i, a_i, r_i, s'_i)$ from $\mathcal{B}$
      Set $a'_i = \arg\min_a Q_\sigma(s'_i, a; \theta)$
      Set $y_i = (r_i - \eta)^2 - \eta_\sigma + Q'_\sigma(s'_i, a'_i; \theta')$
      Update $\theta$ by minimizing the loss
      $L(\theta) = \frac{1}{n} \sum_i (y_i - Q_\sigma(s_i, a_i; \theta))^2$
    **end if**
    **if** $t$ mod $T_{target}$ **then**
      Update $\theta' \leftarrow \theta$
    **end if**
  **end for**
**end for**

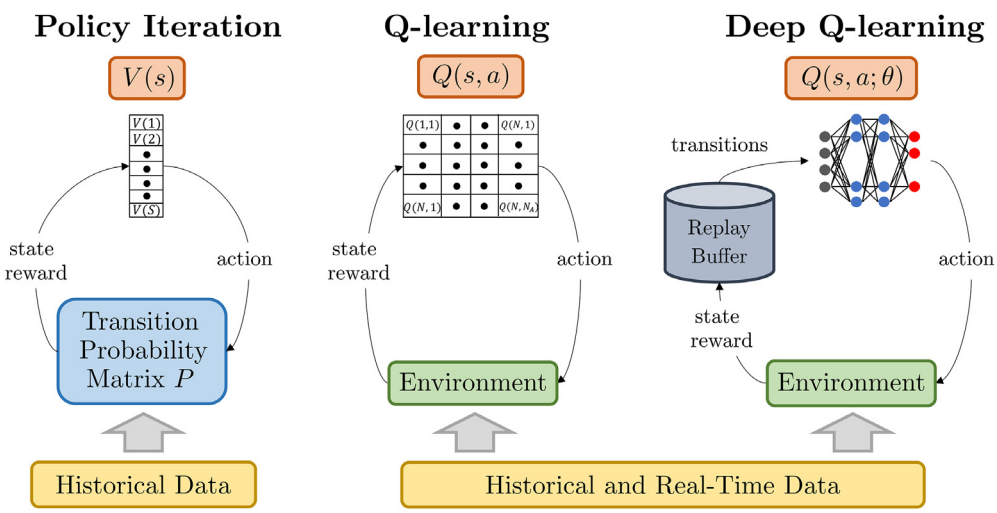



**Fig. 2.** Comparison of the proposed algorithms.

**Table 1**
Parameters of the wind turbine.

| Symbol | Description | Setting |
|---|---|---|
| $V_{cutin}$ | Cut-in speed | 3 m/s |
| $V_{cutout}$ | Cut-out speed | 25 m/s |
| $V_{rated}$ | Rated speed | 9 m/s |
| $W_{cap}$ | Rated power | 3 MW |

**Table 2**
Parameters of the battery storage.

| Symbol | Description | Setting |
|---|---|---|
| $D_{max}$ | Maximum discharge power | 2.4 MW |
| $C_{max}$ | Maximum charge power | 2.4 MW |
| $B_{cap}$ | Battery available capacity | 3 MWh |

**Table 3**
Result comparison of discrete-state experiments.

| Algorithm | Mean/MW | Variance/MW$^2$ |
|---|---|---|
| No battery storage | 0.5942 | 0.8719 |
| Rule-based policy | 0.3281 | 0.0975 |
| Policy iteration | 0.1740 | 0.0389 |
| Tabular Q-learning | 0.1934 ± 0.0013 | 0.0455 ± 0.0005 |

## 4. Numerical experiments

### 4.1. Discrete-state experiments

In this subsection, we compare the performances of the proposed algorithms in discrete cases. In order to set a baseline for comparison, we first study the mean and variance of wind power output without BESS. This would be the worst case since all fluctuations of the wind power, which can be reflected in the variance indicator, will be absorbed by the main network. The rule-based method is easy to implement and is a commonly used algorithm in practice. Therefore, we design a rule-based policy in discrete cases to compare with the policy iteration and Q-learning proposed in this paper.

The system parameters and discrete variables are as follows. Table 1 shows the parameters of the wind turbine, which can be achieved by several Leitwind LTW90 1MW turbines [48]. Table 2 provides the parameters of the BESS. In policy iteration and Q-learning, we store the fluctuation potential and Q-function value in a table with the size $N \times N_A = 2400 \times 119 = 285600$, where the state number $N = N_W \times N_B \times N_T = 10 \times 10 \times 24 = 2400$ and the action number $N_A = N_{a^{[1]}} \times N_{a^{[2]}} = 17 \times 7 = 119$. The setting of the states of the wind power and the battery storage is shown in Appendix.

#### 4.1.1. Rule-based policy

The rule-based policy can be regarded as a simple and practical algorithm in reality, but it only uses the information of current wind power output and battery energy level, without modeling and learning the stochastic characteristics of the system. Fig. 3 shows the flowchart of the rule-based policy for both discrete-state and continuous-state cases. The basic idea of the rule-based policy is that we make the total power output equal to the predefined standard output value $W_{std}$ by charging and discharging the battery storage as possible as we can, while wind curtailment is allowed. When the wind power output $w_t$ is exactly equal to $W_{std}$, the battery operation and wind curtailment are either not required. When $w_t$ is less than $W_{std}$, the battery will discharge as much as possible without abandoning the wind power. When $w_t$ is larger than $W_{std}$, we first charge or discharge the battery storage as much as possible, and then deal with the rest amount of wind power needed to be abandoned. When the remaining wind power $w_t - W_{std} - a_t^{[1]}$ is above the curtailment limitation $W_{cur}$, which is 1.8 MW per hour in this paper, the excess $w_t - W_{std} - a_t^{[1]} - W_{cur}$ will be sent to the main grid. Note that the constraint for maximum battery discharge power $D_{max}$ and charge power $C_{max}$ should be met at the same time. The purpose of the rule-based policy is to achieve the balance between wind curtailment and battery operation potential as far as possible, so as to balance the mean and variance of the total power output. It is shown in Table 3 that when the BESS is integrated into the wind farm, the variance of total power output under the rule-based policy, which is about 0.0975 MW$^2$, is much smaller than that in the case without BESS. At the same time, the mean of total power output decreases to about 0.3281 MW due to wind curtailment operations.

#### 4.1.2. Policy iteration

As for the sensitivity-based policy iteration algorithm, it provides us with the optimal policy representing the mapping from the state space to the action space. By establishing the stochastic Markov model of the system and applying the concept of fluctuation potential to derive the fluctuation-difference formula between different policies, the policy iteration algorithm outperforms the rule-based policy. It can be seen that the mean of total power output is 0.1740 MW and the variance is 0.0389 MW$^2$. Compared with the rule-based policy, the reduction of the mean value indicates that the wind curtailment operation is chosen in more states.

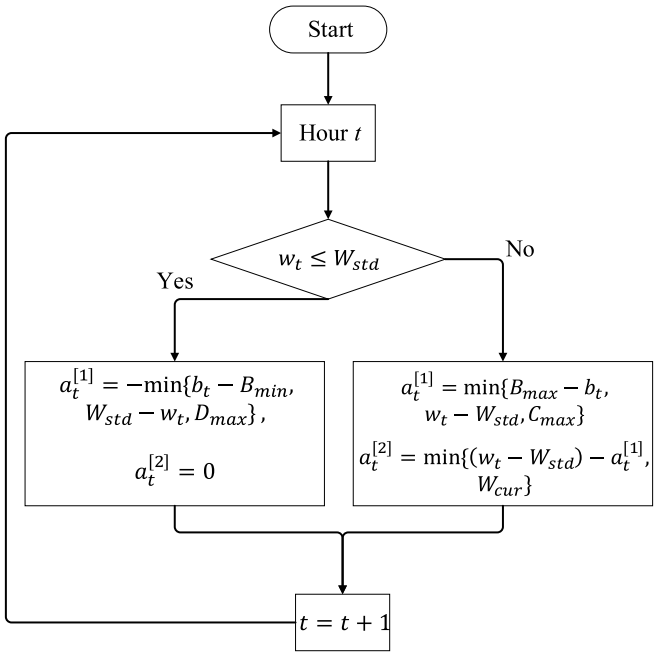



**Fig. 3.** Rule-based policy flowchart.

However, the variance is obviously reduced, benefiting from the reasonable combination of BESS operation and wind curtailment operation.

*4.1.3. Tabular Q-learning*

The Q-learning differs from the policy iteration in that it is model-free and online. The operation policy can be improved by interacting with the environment constantly, and it is not necessary to build a stochastic model for the system in advance. The super-parameters are shown in Table A.7 in Appendix. We run the simulation for 10 times with different random seeds. Sample paths are generated by probability matrix $\boldsymbol{P}$ so that we get a long enough track. Note that this is for improving the experimental results and verifying the effectiveness of the algorithm. In practice, real data can be directly used for training. For each experiment, we test the established policy with a sample path of 10 000 steps after training, then we calculate the mean and variance of total power output. This experiment is performed with Python on a server with 28 processors (10 of 28 are actually used), each of which has an Intel Skylake Xeon Platinum 8163 CPU @ 2.5 GHz. It takes totally about 7.5 h to train.

As shown in Table 3, the variance and mean in Q-learning are about 0.0455 $MW^2$ and 0.1934 MW, which is comparable to that in policy iteration, but slightly worse than policy iteration. It should be noted that due to the different random seeds, the results obtained on the test set are different each time, so the results have deviation. So we show the average and standard deviation of several results in Table 3.

Fig. 4 shows the trajectory of the mean and variance of the total power output during the training of Q-learning in training set. The policy is constantly updated with the increase of training time, and the variance reach the minimum and is nearly stable at the end of the training. The mean fluctuates more violently and shows a trend of decrease, which indicates that the algorithm sacrifice part of the total power output to reduce the variance.

The comparison of the short-term total power output profile between the four cases within 24 h is shown in Fig. 5. As we can see that the worst performance happens when BESS is not applied, and there are clear improvements under the other three algorithms. The performance of the policy iteration algorithm and Q-learning is similar, which is consistent with the long-term performance shown in Table 3. Meanwhile, the policy iteration algorithm and Q-learning are better than the rule-based policy, because they use the information based on system probabilistic characteristics and sample paths, while the rule-based policy only uses the current system information to make judgments.

Fig. 6 presents long-term power output probability distributions in the four cases. The probability distribution histogram can help to compare the probability of extreme power output and equilibrium level of each power output in different algorithms. It can be observed that the probability distributions of policy iteration and Q-learning are more concentrated than that of No-BESS case and the rule-based policy. This can lead to a small variance of total power output in policy iteration algorithm and Q-learning. This conclusion drew from Fig. 6 is also consistent with the figures in Table 3. However, we can also observe that the operating condition seems to be shifting toward zero power output when the proposed method is utilized. To guarantee the policy not to converge to zero power output, we set the limitation of maximal wind curtailment. The combined mean–variance metric can be studied in our further work to penalize the curtailment.

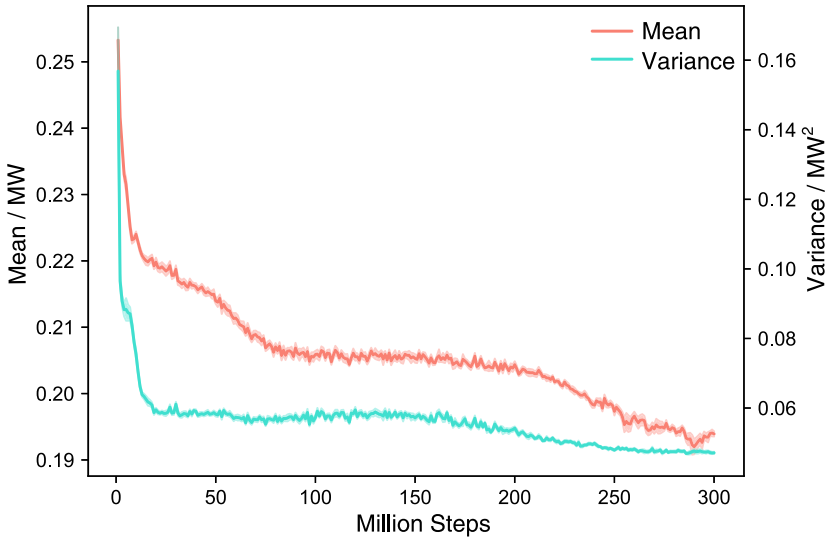



**Fig. 4.** The mean and variance of total power output during the training of tabular Q-learning.

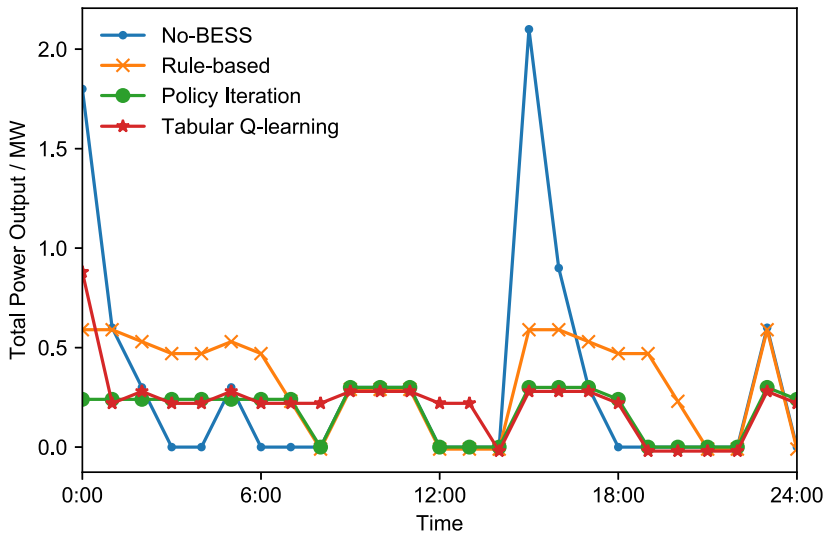



**Fig. 5.** Short-term profile in four discrete-state cases within 24 h.

**Table 4**
Result comparison of continuous-state experiments.

| Algorithm | Mean/MW | Variance/MW$^2$ |
|---|---|---|
| No battery storage | 0.7313 | 1.0198 |
| Rule-based policy | 0.4515 | 0.1521 |
| Deep Q-learning | 0.2472 ± 0.0065 | 0.0842 ± 0.0017 |

### *4.2. Continuous-state experiments*

#### *4.2.1. Rule-based policy*

In the continuous-state rule-based policy, all the variables are continuous, offering the rule-based policy a higher degree of freedom to operate. It is shown in Table 4 that the variance of total power output under the rule-based policy is 0.1521 MW$^2$, which is better than that in No-BESS case.

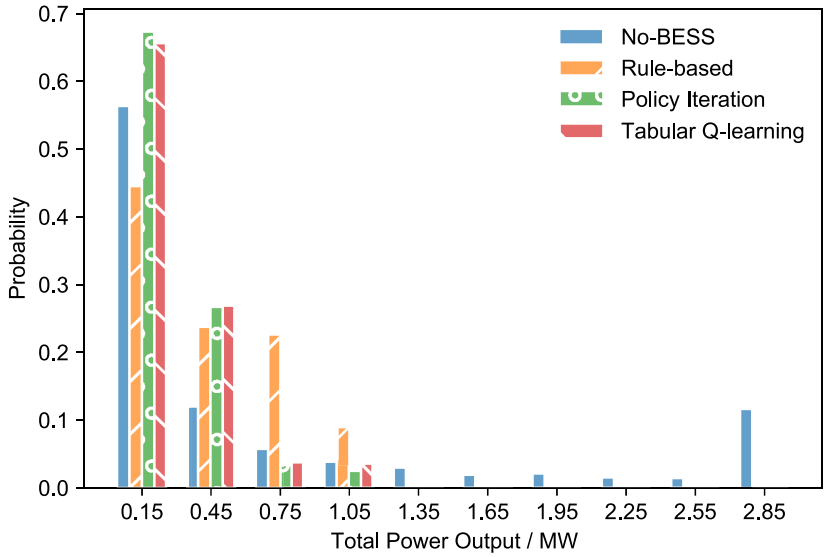



**Fig. 6.** Long-term power output probability distributions in four discrete-state cases.

*4.2.2. Deep Q-learning*

In the continuous-state experiments, we directly use the real data from [41] for training and testing as discussed in Section 2. We extracted 15 years of wind speed profile from the data set. The data extracted is from 0:00 September 1, 2005 to 23:59 September 1, 2020. We reserve the last 10 000 records for testing, and use the rest for training. This experiment is performed with Python on a server with 28 processors (10 of 28 are actually used), each of which has an Intel Skylake Xeon Platinum 8163 CPU @ 2.5 GHz and 4 NVIDIA P100 GPUs. It takes totally about 10 h to train.

Different from the discrete-state cases, the wind power output value and battery storage level is continuous, hence the number of states of the feasible actions are set more than that in discrete-state experiments for a more precise operation.

The neural network structure in the proposed deep Q-learning is a three-layer full-connection one with tanh as the activation function, as shown in Fig. 7. The hidden size is set to be 128. As discussed in Section 3, the last layer of the network has two branches, value branch and advantage branch, because of the application of dueling network. The setting of super-parameters are shown in Table A.8 in Appendix.

Similar to discrete-state experiments, we run the simulation for 10 times with different random seeds and calculate the mean and variance on the test set. It is shown in Table 4 that the variance and mean in deep Q-learning are about 0.0842 $MW^2$ and 0.2472 MW respectively. Note that the results of the rule-based policy and No-BESS policy are deterministic because they are the performance on the entire real data track. Though the trained deep Q-learning policies are deterministic in evaluation, they are influenced by the varied parameter initialization and the random exploration strategy during training. Therefore only the standard deviation of the results in deep Q-learning is shown in Table 4. It can be observed that the deep Q-learning algorithm outperforms the continuous-state rule-based policy on variance, and it also does considerably better than the No-BESS case. It can be observed from Fig. 8 that the variance of total power output shows a trend of decreasing, while the mean increases and decreases substantially first, and then increases slowly to a certain level and fluctuates. Compared with the rule-based algorithm with continuous actions, the actions in deep Q-learning are discrete, which restricts the performance of deep Q-learning to some degree. For all such unfavorable conditions, deep Q-learning provides a better operation policy since the variance is much smaller than that of the other algorithms. We verify that this gain is brought in by the learning capability of deep Q-learning.

The short-term total power output profile within 24 h under the three policies are shown in Fig. 9. The worst case is No-BESS, showing the biggest fluctuation. The application of rule-based policy and deep Q-learning when the BESS is added makes the fluctuation of the total power output decrease obviously, and deep Q-learning outperforms the rule-based policy. It can be observed that from 14:00 to 24:00 the fluctuation is effectively reduced by deep Q-learning compared to the rule-based policy.

Fig. 10 demonstrates the long-term power output probability distribution in the three cases. It can be observed that deep Q-learning performs the best in the ability to reduce the probability of extreme output and to make the probability of various power outputs more similar. What is more, the number of actions may slightly influence the performance and the training process of DQN. More study can be done to evaluate the influence of state space and action space to the algorithm performances.

## 5. Conclusion

In this paper, we focus on reducing the long-term fluctuations of the power output from a wind farm with a BESS. The choice of variance as the fluctuation indicator brings the difficulty of utilizing the standard MDP theory to solve this problem. To overcome

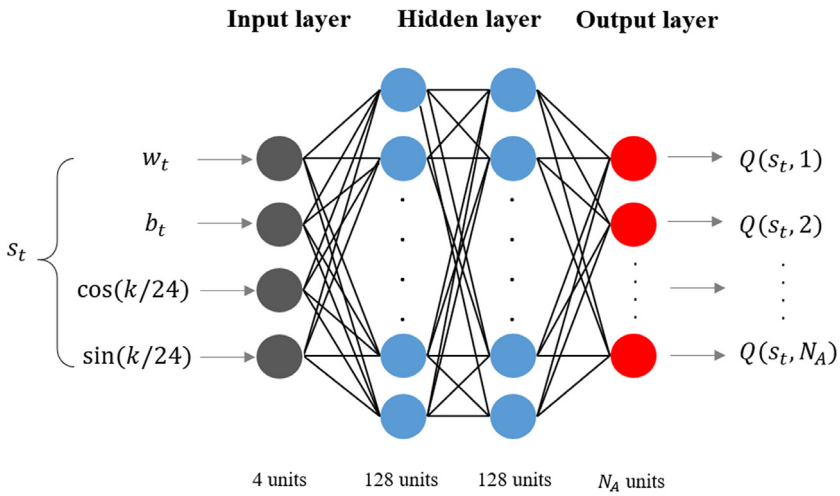



**Fig. 7.** Neural network structure in deep Q-learning.

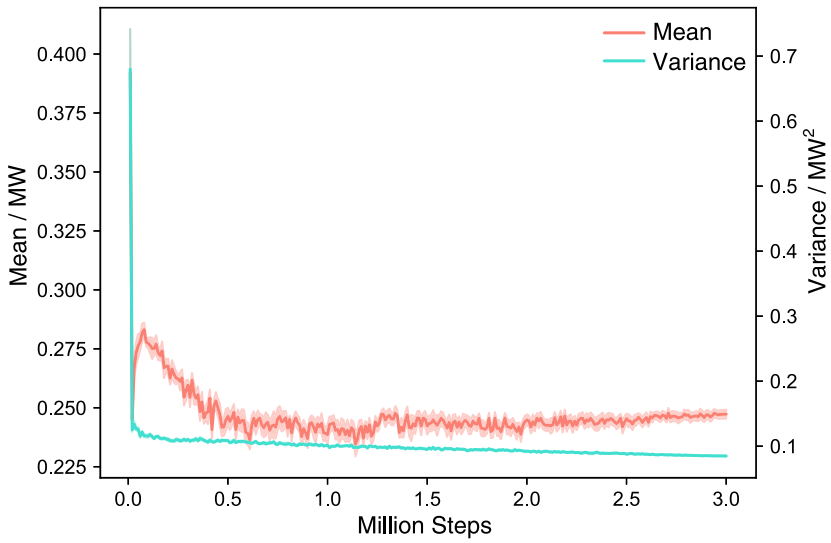



**Fig. 8.** The mean and variance of total power output during the training of deep Q-learning.

this difficulty, we develop a sensitivity-based policy iteration algorithm to obtain the optimal policy of battery scheduling and wind curtailment. Furthermore, we propose the Q-learning and deep Q-learning algorithms for discrete and continuous cases respectively, so that the policy can be improved online by sample paths. The three algorithms proposed in this paper are compared with the corresponding rule-based method which is widely applied in practice. Numerical experiments demonstrate the effectiveness of our proposed algorithms in different cases where the system probability structure is either given or not. In the future, the learning approaches with both mean and variance values as the object function deserve further investigation.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgment

This work was supported in part by the National Key Research and Development Program of China (2017YFC0704100), the National Natural Science Foundation of China (NSFC 62073346, 11931018, U1301254, 61425027), the Guangdong Basic and Applied Basic Research Foundation, China (Grant No. 2021A1515011984), the Guangdong Province Key Laboratory of

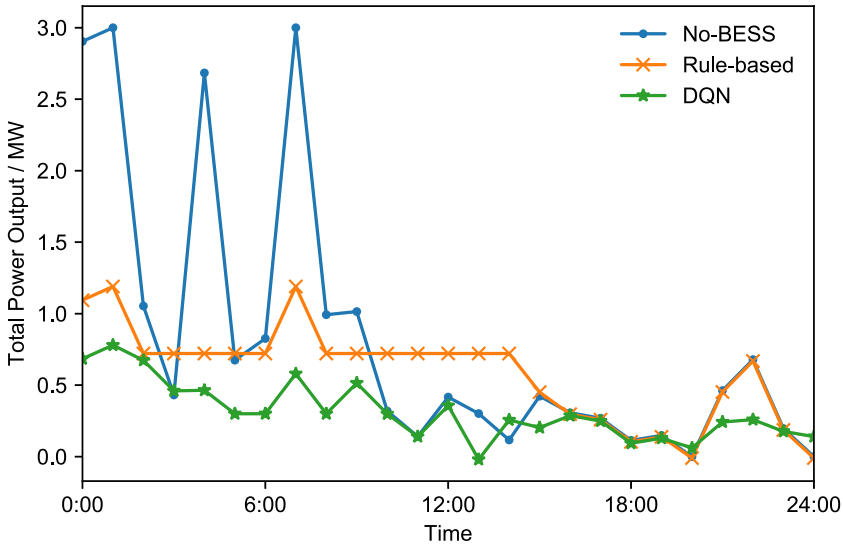



**Fig. 9.** Short-term profile in three continuous-state cases within 24 h on November 24, 2015.

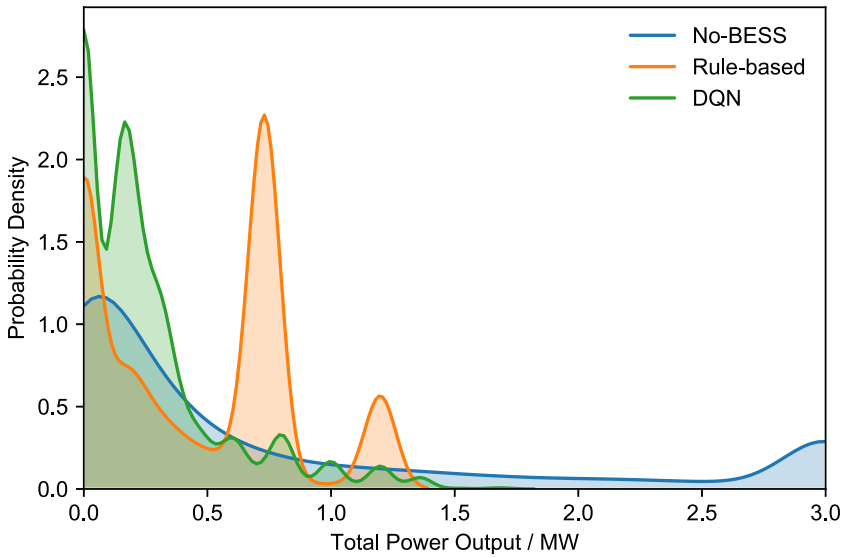



**Fig. 10.** Long-term power output probability distributions in three continuous-state cases.

Computational Science at the Sun Yat-Sen University, China (2020B1212060032), and the National 111 International Collaboration Project, China (BP2018006).

## Appendix

We demonstrate the states of the wind power, the battery storage and the feasible actions for discrete states and continuous states and their corresponding values in the appendix. Besides, the super-parameters of the tabular Q-learning and deep Q-learning algorithms are also listed as follows.

Table A.1 shows the correspondence between the wind power states and the hourly average power output. Note that the biggest discretized wind power output and battery level is smaller than their real value, since we round down in the discretization. Table A.2 shows the correspondence between the battery storage states and battery level. Tables A.3 and A.4 shows the feasible actions for

**Table A.1**
Discrete states of the wind power output.

| State | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| Average output/MW | 0 | 0.3 | 0.6 | 0.9 | 1.2 | 1.5 | 1.8 | 2.1 | 2.4 | 2.7 |

**Table A.2**
Discrete states of the battery storage.

| State | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| Battery level/MWh | 0 | 0.3 | 0.6 | 0.9 | 1.2 | 1.5 | 1.8 | 2.1 | 2.4 | 2.7 |

**Table A.3**
Feasible actions of the bess for discrete states.

| $a^{[1]}$ Action index | 1 | 2 | 3 | 4 | 5 | ⋯ | 9 | ⋯ | 13 | 14 | 15 | 16 | 17 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| BESS operation/MW | −2.4 | −2.1 | −1.8 | −1.5 | −1.2 | ⋯ | 0 | ⋯ | +1.2 | +1.5 | +1.8 | +2.1 | +2.4 |

**Table A.4**
Feasible actions of wind curtailment for discrete states.

| $a^{[2]}$ Action index | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| Wind curtailment operation/MW | 0 | +0.3 | +0.6 | +0.9 | +1.2 | +1.5 | +1.8 |

**Table A.5**
Feasible actions of bess for continuous states.

| $a^{[1]}$ Action index | 1 | 2 | 3 | 4 | 5 | 6 | ⋯ | 20 | 21 | 22 | 23 | 24 | 25 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| BESS operation/MW | −2.4 | −2.2 | −2.0 | −1.8 | −1.6 | −1.4 | ⋯ | 1.4 | 1.6 | 1.8 | 2.0 | 2.2 | 2.4 |

**Table A.6**
Feasible actions of wind curtailment for continuous states.

| $a^{[2]}$ Action index | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| Wind curtailment operation/MW | 0 | +0.2 | +0.4 | +0.6 | +0.8 | +1.0 | +1.2 | +1.4 | +1.6 | +1.8 |

**Table A.7**
Super-parameters of tabular Q-learning.

| Symbol | Description | Setting |
|---|---|---|
| $\alpha$ | Learning rate | 0.001 |
| $\beta$ | Smooth factor | 0.01 |
| $\epsilon$ | Exploration factor | 0.5 |
| $T$ | Episode length for training | 2000 |
| $T_{eval}$ | Episode length for evaluation | 200 |
| $M_{eval}$ | Episodes for evaluation | 100 |

**Table A.8**
Super-parameters of deep Q-learning.

| Symbol | Description | Setting |
|---|---|---|
| $\alpha$ | Learning rate | 1e−4 |
| $\beta$ | Smooth factor for | 1e−2 |
| $\epsilon$ | Exploration factor | 0.03 |
| $n$ | Batch size | 128 |
| $\|B\|$ | Replay buffer size | 1e6 |
| $T$ | Episode length for training | 2000 |
| $T_{eval}$ | Episode length for evaluation | 200 |
| $T_{update}$ | Updating steps | 1 |
| $T_{target}$ | Target replacing steps | 200 |
| $M_{eval}$ | Episodes for evaluation | 10 |

discrete cases. Tables A.5 and A.6 show the correspondence between the states and real values for continuous cases. Tables A.7 and A.8 show the super-parameters of the tabular Q-learning and deep Q-learning algorithms.

## References


[1] Murdock HE, Gibb D, André T, Appavou F, Brown A, Epp B, et al. Renewables 2019 global status report. 2019.
[2] Blavette A, O'Sullivan DL, Alcorn R, Lewis TW, Egan MG. Impact of a medium-size wave farm on grids of different strength levels. IEEE Trans Power Syst 2013;29(2):917–23.
[3] Tang X, Sun Y, Zhou G, Miao F. Coordinated control of multi-type energy storage for wind power fluctuation suppression. Energies 2017;10(8):1212.
[4] Nguyen TNH, Yabe K, Ito M, Dao VT, Ishii H, Hayashi Y. Spinning reserve quantification considering confidence levels of forecast in systems with high wind and solar power penetration. IEEJ Trans Electr Electron Eng 2019;14(9):1304–13.
[5] Qi W, Liang Y, Shen Z-JM. Joint planning of energy storage and transmission for wind energy generation. Oper Res 2015;63(6):1280–93.
[6] Yang Z, Xia L, Guan X. Fluctuation reduction of wind power and sizing of battery energy storage systems in microgrids. IEEE Trans Autom Sci Eng 2020;17(3):1195–207.
[7] Zhao X, Yan Z, Xue Y, Zhang X-P. Wind power smoothing by controlling the inertial energy of turbines with optimized energy yield. IEEE Access 2017;5:23374–82.
[8] Zhou Y, Yan Z, Li N. A novel state of charge feedback strategy in wind power smoothing based on short-term forecast and scenario analysis. IEEE Trans Sustain Energy 2016;8(2):870–9.
[9] Bai L, Li F, Hu Q, Cui H, Fang X. Application of battery-supercapacitor energy storage system for smoothing wind power output: An optimal coordinated control strategy. In: 2016 IEEE power and energy society general meeting (PESGM). Boston, USA, 17-21 July; 2016, p. 1–5.
[10] Jiang Q, Wang H. Two-time-scale coordination control for a battery energy storage system to mitigate wind power fluctuations. IEEE Trans Energy Convers 2012;28(1):52–61.
[11] Zhang Z, Zhang Y, Lee W-J. Energy storage based optimal dispatch scheme for financial improvement and fluctuation mitigation on wind power generation. In: 2017 IEEE industry applications society annual meeting. Cincinnati, USA, 1-5 October; 2017, p. 1–7.
[12] Quan H, Srinivasan D, Khosravi A. Incorporating wind power forecast uncertainties into stochastic unit commitment using neural network-based prediction intervals. IEEE Trans Neural Netw Learn Syst 2015;26(9):2123–35.
[13] Zhai Q, Li X, Lei X, Guan X. Transmission constrained UC with wind power: An all-scenario-feasible MILP formulation with strong nonanticipativity. IEEE Trans Power Syst 2016;32(3):1805–17.
[14] Shao C, Wang X, Shahidehpour M, Wang X, Wang B. Security-constrained unit commitment with flexible uncertainty set for variable wind power. IEEE Trans Sustain Energy 2017;8(3):1237–46.
[15] Tang C, Xu J, Tan Y, Sun Y, Zhang B. Lagrangian relaxation with incremental proximal method for economic dispatch with large numbers of wind power scenarios. IEEE Trans Power Syst 2019;34(4):2685–95.
[16] Quan H, Khosravi A, Yang D, Srinivasan D. A survey of computational intelligence techniques for wind power uncertainty quantification in smart grids. IEEE Trans Neural Netw Learn Syst 2020;31(11):4582–99.
[17] Wang R, Li C, Fu W, Tang G. Deep learning method based on gated recurrent unit and variational mode decomposition for short-term wind power interval prediction. IEEE Trans Neural Netw Learn Syst 2020;31(10):3814–27.
[18] Teleke S, Baran ME, Bhattacharya S, Huang AQ. Rule-based control of battery energy storage for dispatching intermittent renewable sources. IEEE Trans Sustain Energy 2010;1(3):117–24.
[19] Oriti G, Julian AL, Peck NJ. Power-electronics-based energy management system with storage. IEEE Trans Power Electron 2015;31(1):452–60.
[20] Arcos-Aviles D, Pascual J, Marroyo L, Sanchis P, Guinjoan F. Fuzzy logic-based energy management system design for residential grid-connected microgrids. IEEE Trans Smart Grid 2018;9(2):530–43.
[21] Bai L, Hu Q, Li F, Tao D, Sun H. Robust mean-variance optimization model for grid-connected microgrids. In: IEEE power & energy society general meeting. Denver, USA, 26-30 July; 2015, p. 1–5.
[22] Saejia M, Ngamroo I. Alleviation of power fluctuation in interconnected power systems with wind farm by SMES with optimal coil size. IEEE Trans Appl Supercond 2012;22(3):5701504.
[23] Li YZ, Wu QH, Li MS, Zhan JP. Mean-variance model for power system economic dispatch with wind power integrated. Energy 2014;72(7):510–20.
[24] Xia L. Optimization of Markov decision processes under the variance criterion. Automatica 2016;73(73):269–78.
[25] Xia L. Variance minimization of parameterized Markov decision processes. Discrete Event Dyn Syst 2018;28(1):63–81.
[26] Cao X-R. Stochastic learning and optimization: A sensitivity-based approach. Springer Science; 2007.
[27] Sutton RS, Barto AG. Reinforcement learning: An introduction. MIT press; 2018.
[28] Guo W, Liu F, Si J, He D, Harley R, Mei S. Online supplementary ADP learning controller design and application to power system frequency control with large-scale wind energy integration. IEEE Trans Neural Netw Learn Syst 2016;27(8):1748–61.
[29] Liu W, Zhuang P, Liang H, Peng J, Huang Z. Distributed economic dispatch in microgrids based on cooperative reinforcement learning. IEEE Trans Neural Netw Learn Syst 2018;29(6):2192–203.
[30] Xiong R, Cao J, Yu Q. Reinforcement learning-based real-time power management for hybrid energy storage system in the plug-in hybrid electric vehicle. Appl Energy 2018;211:538–48.
[31] Vázquez-Canteli JR, Nagy Z. Reinforcement learning for demand response: A review of algorithms and modeling techniques. Appl Energy 2019;235:1072–89.
[32] Wang Z, Hong T. Reinforcement learning for building controls: The opportunities and challenges. Appl Energy 2020;269:115036.
[33] Nyong-Bassey BE, Giaouris D, Patsios C, Papadopoulou S, Papadopoulos AI, Walker S, et al. Reinforcement learning based adaptive power pinch analysis for energy management of stand-alone hybrid energy storage systems considering uncertainty. Energy 2020;193:116622.
[34] Liu H, Yu C, Wu H, Duan Z, Yan G. A new hybrid ensemble deep reinforcement learning model for wind speed short term forecasting. Energy 2020;117794.
[35] Zakaria A, Ismail FB, Lipu MH, Hannan MA. Uncertainty models for stochastic optimization in renewable energy applications. Renew Energy 2020;145:1543–71.
[36] Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, et al. Human-level control through deep reinforcement learning. Nature 2015;518(7540):529.
[37] Bui V-H, Hussain A, Kim H-M. Double deep Q-learning-based distributed operation of battery energy storage system considering uncertainties. IEEE Trans Smart Grid 2019;11(1):457–69.
[38] Bui V-H, Hussain A, Kim H-M. Q-learning-based operation strategy for community battery energy storage system (CBESS) in microgrid system. Energies 2019;12(9):1789.
[39] Huang Q, Jia Q-S, Guan X. A multi-timescale and bilevel coordination approach for matching uncertain wind supply with EV charging demand. IEEE Trans Autom Sci Eng 2016;14(2):694–704.
[40] Richmond N, Jacko P, Makowski AM. Optimal planning of slow-ramping power production in energy systems with renewables forecasts and limited storage. In: 2014 international conference on probabilistic methods applied to power systems (PMAPS). Durham, United Kingdom, 7-10 July; 2014, p. 1–6.
[41] Jager D, Andreas A. NREL National Wind Technology Center (NWTC): M2 Tower; Boulder, Colorado (Data); NREL Report No. DA-5500-56489. 1996. http://dx.doi.org/10.5439/1052222.
[42] Puterman ML. Markov decision processes.: Discrete stochastic dynamic programming. John Wiley & Sons; 2014.
[43] Xia L, Glynn PW. A generalized fundamental matrix for computing fundamental quantities of Markov systems. 2016, arXiv preprint arXiv:1604.04343.
[44] Abounadi J, Bertsekas D, Borkar VS. Learning algorithms for Markov decision processes with average cost. SIAM J Control Optim 2001;40(3):681–98.

[45] Mnih V, Kavukcuoglu K, Silver D, Graves A, Antonoglou I, Wierstra D, et al. Playing Atari with deep reinforcement learning. Comput Sci 2013.
[46] Van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double Q-learning. In: AAAI conference on artificial intelligence, Vol. 2. Phoenix, USA, 12-17 February; 2016, p. 5.
[47] Wang Z, Schaul T, Hessel M, Hasselt H, Lanctot M, Freitas N. Dueling network architectures for deep reinforcement learning. In: International conference on machine learning (ICML). New York, USA, 19-24 June; 2016, p. 1995–2003.
[48] Leitwind LTW90 1000 [Online] https://en.wind-turbine-models.com/turbines/1655-leitwind-ltw90-1000.